

1 L2–3 Supervised Learning

1.1 Classification and Loss

ERM: population risk $\theta^* = \arg \min_{\theta} \mathbb{E}_{x \sim p} \text{err}(f(x; \theta), y(x))$; empirical risk $\hat{\theta} = \arg \min_{\theta} \frac{1}{N} \sum_i \text{err}(f(x_i; \theta), y_i)$. **Binary logistic**: $p(y = 1|x) = \sigma(w^\top x + b)$,

$$\ell = -y \log p - (1 - y) \log(1 - p), \quad \frac{\partial \ell}{\partial z} = p - y.$$

Multi-class: logits $z \in \mathbb{R}^K$, $p_k = e^{z_k} / \sum_j e^{z_j}$,

$$\ell(z, y) = -\log p_y = -z_y + \log \sum_j e^{z_j}, \quad \frac{\partial \ell}{\partial z_k} = p_k - 1[k = y].$$

One-hot CE: $-\sum_k y_k \log p_k$.

1.2 MLP and Backprop

Layer: $h^{(0)} = x$, $z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$, $h^{(\ell)} = \phi(z^{(\ell)})$. Batch code convention often uses $X \in \mathbb{R}^{B \times n}$, $Z = XW + b$, $W \in \mathbb{R}^{n \times m}$. **Backprop**: define $\delta^{(\ell)} = \partial L / \partial z^{(\ell)}$:

$$\partial_{W^{(\ell)}} L = \delta^{(\ell)} (h^{(\ell-1)})^\top, \quad \partial_{b^{(\ell)}} L = \delta^{(\ell)},$$

$$\delta^{(\ell-1)} = ((W^{(\ell)})^\top \delta^{(\ell)}) \odot \phi'(z^{(\ell-1)}).$$

Action: sigmoid/tanh saturate; ReLU $\max(0, z)$ keeps positive-side gradient; ELU/smooth variants improve negative side. **Subgradient**: ReLU at 0 uses any value in $[0, 1]$ by convention.

1.3 CNN

Motivation: locality + weight sharing. Convolution:

$$y_{i,j,\rho} = \sum_{u,v,c} K_{u,v,c,\rho} x_{i+u,j+v,c} + b_\rho.$$

For input $H \times W \times C_{\text{in}}$, kernel $K_k \times K_w$, padding P , stride S :

$$H' = \lfloor (H + 2P - K_h) / S \rfloor + 1, \quad W' = \lfloor (W + 2P - K_w) / S \rfloor + 1.$$

Params $K_h K_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$. **Scanning MLP**: same detector slides across space. **Pooling**: max/avg reduces spatial size and jitter. **Padding**: keeps boundary info; same padding for odd kernel uses $P = (K - 1) / 2$.

1.4 Optimization

GD: $x_{k+1} = x_k - \eta_k \nabla f(x_k)$. If ∇f is L -Lipschitz:

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2} \|y - x\|^2,$$

so $0 < \eta < 2/L$ decreases the upper bound. **Newton**: from quadratic Taylor,

$$x_{k+1} = x_k - \eta [\nabla^2 f(x_k)]^{-1} \nabla f(x_k).$$

g_k = gradient on x_k . **Momentum**: $v_{k+1} = \beta v_k + g_k$, $x_{k+1} = x_k - \eta v_{k+1}$. **Nesterov**: compute gradient at lookahead $x_k + \beta(x_k - x_{k-1})$. **AdaGrad**: $s_k = s_{k-1} + g_k^2$, $x_{k+1} = x_k - \eta g_k / (\sqrt{s_k} + \epsilon)$; may decay too fast. **RM-SProp**: $s_k = \beta s_{k-1} + (1 - \beta) g_k^2$. **Adam**:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k, \quad v_k = \beta_2 v_{k-1} + (1 - \beta_2) \hat{s}_k^2,$$

$$x_{k+1} = x_k - \eta \hat{m}_k / (\sqrt{\hat{v}_k} + \epsilon), \quad \hat{m}_k = m_k / (1 - \beta_1^k).$$

1.5 Regularization and Architectures

Dropout: randomly zero units, approximates ensemble and reduces co-adaptation. **Weight decay**: add $\lambda \|\theta\|^2$. **Gradient clipping**: $g \leftarrow g \min(1, \tau / \|g\|)$.

BatchNorm: $\mu_B = \frac{1}{B} \sum_i z_i$, $\sigma_B^2 = \frac{1}{B} \sum_i (z_i - \mu_B)^2$, $z_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$.

LayerNorm: normalize hidden dimension inside one sample; better for sequence models. **ResNet**: $h_{t+1} = h_t + F(h_t)$, residual path eases optimization. **DenseNet**: concatenate previous features. **FCN**: replace fully connected layer by convolution for dense prediction.

2 L4 Energy-Based Models

2.1 Energy View

Generative modeling: learn $P(X)$ or $P(X, y) = P(y)P(X|y)$ instead of $P(y|X)$. **EBM**: unnormalized score via energy

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}, \quad Z_\theta = \int e^{-E_\theta(x)} dx.$$

Low energy means high probability; training and inference both need sampling or partition-function gradients. $\log p_\theta(x) = -E_\theta(x) - \log Z_\theta$, $\nabla_\theta \log p_\theta(x) = -\nabla_\theta E_\theta(x) - \frac{1}{Z_\theta} \nabla_\theta \left(\sum_{x'} e^{-E_\theta(x')} \right) = -\nabla_\theta E_\theta(x) + \mathbb{E}_{x' \sim p_\theta} [E_\theta(x')]$.

2.2 Hopfield Network

State $y_i \in \{-1, 1\}$, local field $h_i = \sum_{j \neq i} w_{ij} y_j + b_i$. Update flips if $y_i h_i < 0$.

$$E(y) = -\sum_{i < j} w_{ij} y_i y_j - \sum_i b_i y_i = -\frac{1}{2} y^\top W y - b^\top y.$$

Asynchronous flip changes energy by $\Delta E = 2y_i h_i$; if $y_i h_i < 0$, energy decreases. Finite states imply convergence to a local minimum. **Hebbian storage**: $w_{ij} = \frac{1}{P} \sum_p y_i^p y_j^p$, $w_{ii} = 0$. Multiple patterns cause **spurious minima**; capacity is limited, roughly linear in neuron count for stable random memories.

Objective for W : $W = \arg \min_W (\sum_{\mathbb{C} \in P} E(y) - \sum_{\mathcal{L} \in P} E(y'))$. **Optimization view**: desired patterns should be valleys; short-run evolution can raise undesired valleys by contrastive updates.

SGD update rule for W : $W \leftarrow W - \eta \left(\mathbb{E}_{y \in P} [y y^\top] - \mathbb{E}_{y' \sim p_\theta} [y' y'^\top] \right)$, initialize y' by a random desired pattern (in P), run evolution for a few timesteps.

2.3 Boltzmann Machine

Stochastic Hopfield: energy $E(y)$ as Hopfield, $p(y) = \frac{1}{2} e^{-E(y)} \propto e^{-E(y)}$.

Conditional Gibbs update for $\{-1, 1\}$ units:

$$P(y_i = 1 | y_{-i}) = \frac{1}{1 + \exp(-\sum_j w_{ij} y_j - 2b_i)} = \frac{1}{1 + e^{-2h_i}}.$$

Temperature T : $p_T(y) \propto e^{-E(y)/T}$; annealing lowers T from exploration to exploitation.

BM Training: maximize $L(W) = \sum_{y \in P} \log p_\theta(y)$, θ means parameters. $\nabla w_{ij} \log p_\theta(y) = y_i y_j - \mathbb{E}_{p_\theta} [y_i y_j]$, $\nabla w_{ij} L = \mathbb{E}_{y \in P} [y_i y_j] - \mathbb{E}_{y' \sim p_\theta} [y'_i y'_j] = \left[\frac{1}{P} \sum_{y \in P} y_i y_j - \mathbb{E}_{y' \sim p_\theta} [y'_i y'_j] \right]$. First term: compute average over P . Second term: sample from p_θ , compute average. **Sample method**: random initialize $y(0)$, cycle over i (coordinates) and sample from $P(y_i(t) | y_{-i}(t))$, after convergence select the final M states as samples. **Updating**: $w_{ij} \leftarrow w_{ij} + \eta \nabla w_{ij} L(W)$. Learning = **positive phase** data correlation minus **negative phase** model correlation.

With hidden neurons: $y = (v, h)$

$$\nabla w_{ij} L = \frac{1}{|P|} \sum_{v \in P} \mathbb{E}_h [y_i y_j] - \mathbb{E}_{y' \sim p(v, h)} [y'_i y'_j].$$

Boltzmann for classification: $y = (v, h, c)$, c : visible, one-hot vector

2.4 RBM and Contrastive Divergence

RBM: bipartite v, h ; no visible-visible or hidden-hidden edges: $E(v, h) = -v^\top W h - b^\top v - c^\top h$. Conditionals factorize: $P(h_j = 1 | v) = \sigma(c_j + \sum_i w_{ij} v_i)$, $P(v_i = 1 | h) = \sigma(b_i + \sum_j w_{ij} h_j)$, $\sigma(z) = \frac{1}{1 + e^{-z}}$. Sampling: $v^{(0)} \rightarrow h^{(0)} \rightarrow v^{(1)} \rightarrow h^{(1)} \rightarrow \dots$

Maximum likelihood estimate: $\nabla w_{ij} L = \frac{1}{N_P} \sum_{v \in P} v_0 h_{0j} - \frac{1}{M} \sum_{v \sim h} v_0 h_{0j}$.

Only 3 gibbs sampling steps needed: $\nabla w_{ij} L = \frac{1}{N_P} \sum_{v \in P} v_0 h_{0j} - v_{1i} h_{1j}$.

CD-k: start at data $v^{(0)}$, run k Gibbs steps, update $W \propto v^{(0)} h^{(0)\top} - v^{(k)} h^{(k)\top}$.

2.5 Sampling

Inverse CDF: sample $u \sim U[0, 1]$, choose category by cumulative mass. **Box-Muller**: Gaussian from uniform:

$$z_1 = \sqrt{-2 \log u_1} \cos(2\pi u_2), \quad z_2 = \sqrt{-2 \log u_1} \sin(2\pi u_2).$$

Importance sampling: $\mathbb{E}_p[f(x)] = \mathbb{E}_q[f(x)p(x)/q(x)]$. Good q must cover p 's high-mass region. **MCMC**: **Detailed balance** $\pi(x)T(x \rightarrow x') = \pi(x')T(x' \rightarrow x)$ implies stationarity of π . **Ergodic**: (intuitively) you can visit any desired state with positive probability from any state, ensure a unique stationary distribution. **Metropolis Hastings Algo.**: Construct Markov chain with stationary π (desired distribution). Current x , draw $x' \sim q(x'|x)$ (called proposal), transit to x' with prob. $\alpha = \min \left(1, \frac{\pi(x')q(x|x')}{\pi(x)q(x'|x)} \right)$, otherwise

stays at x . Choice of q : random proposal $q(x'|x) = \text{noise}$ (e.g. Gaussian, then it relies on $|x - x'|$), then q terms in α vanishes.

For EBM, partition Z cancels.

Gibbs: sample a single coordinate per step. Acceptance ratio always 1.

SGLD: $x_{t+1} = x_t - \frac{\eta}{2} \nabla_x E_\theta(x_t) + \sqrt{\eta} \xi_t$.

3 L5 Generative Adversarial Networks

3.1 Implicit Generative Model

Instead of density $p_\theta(x)$, learn sampler $x = G_\theta(z)$, $z \sim p_z$. Likelihood-free learning. Need differentiable distance between generated and real distributions. GAN learns this distance with a discriminator.

GAN: generator $G_\theta(z)$, $z \sim p(z) = N(0, I)$; discriminator $D_\phi(x)$, classify the data is from P_{data} or G_θ . Objective: $L = \min_\theta \max_\phi V(D_\phi, G_\theta)$,

$$V(D_\phi, G_\theta) = \mathbb{E}_{x \sim p_{\text{data}}} \log D_\phi(x) + \mathbb{E}_{z \sim p_z} \log(1 - D_\phi(G_\theta(z))).$$

Dataset: $\{(x, 1) : x \sim p_{\text{data}}\} \cup \{(\tilde{x}, 0) : \tilde{x} \sim G_\theta(z)\}$.

Train D : $L(\theta) = \mathbb{E}_{x \sim p_{\text{data}}} \log D_\phi(x) + \mathbb{E}_{z \sim p_z} \log(1 - D_\phi(G(z)))$.

Train G : $L(\theta) = \mathbb{E}_{z \sim p(z)} [\log D_\phi(G_\theta(z))]$.

For fixed G , the optimal D is $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$. Substitute: $V(D^*, G) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \| p_G)$, **JSD**: $\text{JSD}(p \| q) = \text{KL}(p \| \frac{p+q}{2}) + \text{KL}(q \| \frac{p+q}{2})$, ≥ 0 , $\sqrt{\text{JSD}}$ satisfies triangular inequality

Non-saturating generator loss $-\mathbb{E}_z \log D(G(z))$ gives stronger gradients.

3.2 Evaluation and Failure Modes

Mode collapse: generator covers few modes (converges to a few samples). Intrinsic issue of JSD. **Training instability**: moving target game; discriminator too strong $\Rightarrow$ weak generator gradient. **IS**: rewards confident/diverse ImageNet predictions, but blind to intra-class artifacts. **FID**: Compute μ_P, Σ_P and μ_G, Σ_G for $P_{\text{data}}, G(z)$ using inception v3 pool3 layer (2048-d). Compute Wasserstein distance between two Gaussians. compare Gaussian feature statistics: $\|\mu_P - \mu_G\|^2 + \text{tr}(\Sigma_P + \Sigma_G - 2(\Sigma_P \Sigma_G)^{1/2})$.

3.3 WGAN Family

JSD can be flat when supports do not overlap. Wasserstein-1: $W(p, q) = \sup \|f\|_L \leq 1 \mathbb{E}_{x \sim p} f(x) - \mathbb{E}_{x \sim q} f(x)$.

WGAN: critic f_ϕ , no D . WGAN objective: subject to $\|f_\phi(x)\|_L \leq 1$, $L(\phi) = \mathbb{E}_{x \sim p_{\text{data}}} [f_\phi(x)] - \mathbb{E}_{x \sim p_G} [f_\phi(x)]$, $L(\theta) = \mathbb{E}_{x \sim G_\theta(z)} [f_\phi(x)]$.

Lipschitz constraint: weight clipping, spectral norm, or **WGAN-GP**: $\tilde{x} \leftarrow (1 - \epsilon)x + \epsilon \tilde{x}$, $\epsilon \sim \text{Unif}(0, 1)$, $L(\phi) = \mathbb{E}_{x \sim p_{\text{data}}} [f_\phi(x)] - \mathbb{E}_{\tilde{x} \sim p_G} [f_\phi(x)] + \lambda \mathbb{E}_{\tilde{x}} (\|\nabla_{\tilde{x}} f_\phi(\tilde{x})\|_2 - 1)^2$.

DCGAN: conv/deconv, batchnorm, stable architecture. **BigGAN**: scale + class conditioning + truncation trick. **GigaGAN**: text-to-image GAN scaling. **R3GAN**: regularized robust objective.

3.4 Variants and Applications

Conditional GAN: $G(z, c)$, $D(x, c)$. **Semi-supervised GAN**: discriminator has $K + 1$ classes, K real labels plus fake. **BIGAN/ALI**: learn encoder $E(x)$ with discriminator over (x, z) pairs. Idea: $(z, G(z))$ and $(E(x), x)$ from same distribution. gives representation learning. **Pix2Pix**: paired image-to-image translation with conditional GAN + reconstruction loss. **CycleGAN**: unpaired translation using cycle consistency $G : X \rightarrow Y$, $F : Y \rightarrow X$, $\|F(G(x)) - x\| + \|G(F(y)) - y\|$. **DragGAN**: interactive image manipulation by optimizing latent/code with handle-point constraints.

4 L6 Flow and VAE

4.1 Normalizing Flow

Invertible differentiable generator $x = G_\theta(z)$, transform a simple distribution to the data distribution. $\log p_X(x) = \log p_Z(G^{-1}(x)) + \log \det \frac{\partial G^{-1}}{\partial x}$.

Composition: $z_K = G(z_0) = f_K \circ \dots \circ f_1(z_0)$, $\log p_X(z_K) = \log p_Z(z_0) - \sum_k \log |\det \frac{\partial f_k}{\partial z_k}|$. Flow design needs invertible layer, efficient inverse, efficient

compute determinant of Jacobi.

Coupling: split $x = (x_a, x_b)$, to (y_a, y_b) , $y_a = x_a$, $y_b = x_b \odot e^{S(x_a)} + t(x_a)$, $\log |\det J| = \sum s(x_a)$, s_t : learnable. sometimes needs premutaion among coordinates or linear transformation.

Glow: invertible 1×1 convolution mixes channels; LU decomposition makes determinant cheap: $\log |\det W| = \sum_i \log |u_{ii}|$. **Dequantization**: add uniform noise to discrete pixels so continuous density is well-defined. **Continuous NF / Neural ODE**: $\frac{dz(t)}{dt} = f_\theta(z(t), t)$, $\log p(z(t_1)) = \log p(z(t_0)) - \int_{t_0}^{t_1} \text{tr} \left(\frac{\partial f_\theta}{\partial z(t)} \right) dt$.

4.2 EM and Variational Inference

Latent-variable likelihood $\log p_\theta(x) = \log \sum_z p_\theta(x, z)$. Jensen:

$$\log p_\theta(x) = \log \mathbb{E}_q \frac{p_\theta(x, z)}{q(z)} \geq \mathbb{E}_q \log p_\theta(x, z) + H(q) = \mathcal{L}(q, \theta).$$

Gap: $\log p_\theta(x) - \mathcal{L}(q, \theta) = \text{KL}(q(z) \| p_\theta(z|x))$. EM: E-step $q(z) = p_{\text{gold}}(z|x)$, M-step maximize $\mathbb{E}_q \log p_\theta(x, z)$.

4.3 VAE

Encoder $q_\phi(z|x)$, decoder $p_\theta(x|z)$, prior $p(z)$.

ELBO: $\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \| p(z))$. In practice, $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$, $p_\theta(x|z) = \mathcal{N}(\mu_\theta(z), \beta I)$. Reparameterization: $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$. KL of Gaussian can be computed, here $\text{KL}(\mathcal{N}(\mu, \sigma^2 I) \| \mathcal{N}(0, I)) = \frac{1}{2} \sum_i (\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1)$.

Terms: reconstruction = data fidelity; KL = latent regularization. **Pixel-VAE/NVAE**: hierarchical latent variables improve global coherence and likelihood.

5 L7 Diffusion and Flow Matching

5.1 DDPM

Forward: $q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_{t-1}, \beta_t I)$, $\beta_t = 1 - \alpha_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. Closed form: $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$, $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$. Reverse: use $p_\theta(x_{t-1} | x_t)$ to estimate the prob, $p_\theta(0 : T) = p(x_T) \prod_t p(x_t | x_{t-1} | x_T)$, $p(x_T)$ is gaussian prior. ELBO $\log p_\theta(x_0) \geq \mathbb{E}_{x_1, T \sim q(\cdot | x_0)} [\log \frac{p_\theta(x_0, T)}{q(x_1, T | x_0)}]$, use markovian and use tractable $q(x_{t-1} | x_t, x_0)$: $L = \mathbb{E}_q [\log \frac{p(x_T)}{q(x_T | x_0)}] + \mathbb{E}_q [\log p_\theta(x_0 | x_1)] + \sum_{t=2}^T \text{KL}(q(x_{t-1} | x_t, x_0) \| p_\theta(x_{t-1} | x_t))$ (core of DDPM).

Posterior $q(x_{t-1} | x_t, x_0) = \frac{1}{q(x_t | x_0)} q(x_t | x_{t-1}, x_0) q(x_{t-1} | x_0)$, which is a

Gaussian $N(\bar{\mu}_t, \bar{\beta}_t I)$. Let $\tilde{x}_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$, $\bar{\mu}_t = \frac{\sqrt{\bar{\alpha}_t} - 1}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_t) - 1}{1 - \bar{\alpha}_t} x_t = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{1 - \bar{\alpha}_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$, $\bar{\beta}_t = \frac{1 - \bar{\alpha}_t}{1 - \bar{\alpha}_t} \beta_t$. Predict noise $\epsilon_\theta(x_t, t)$: $p_\theta(x_{t-1} | x_t) = \mathcal{N}(\mu_\theta(x_t, t), \sigma_\theta^2 I)$, $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{1 - \bar{\alpha}_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$. Training: sample x_0 from data, sample timestep t , add noise ϵ , predict $\epsilon_\theta(x_t, t)$. Loss:

$$L = \mathbb{E}_{x_0 \sim p_{\text{data}}, t \sim [1, T], \epsilon \sim \mathcal{N}(0, I)} \|\epsilon_\theta(x_t, t) - \epsilon\|_2^2, \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

$$\text{Sampling: } x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t \epsilon.$$

5.2 Score-Based Models

Score: learn score $\nabla_x \log p(x)$. **Langevin dynamics**: use only score. $x_{t+1} = x_t + \eta \nabla_x \log p(x_t) + \sqrt{2\eta} \epsilon_t$, $\epsilon_t \sim \mathcal{N}(0, I)$. continuous form: $dx_t = \nabla_x \log p(x_t) dt + \sqrt{2dw_t}$, $dw \sim \mathcal{N}(0, dt)$. **NCSN**: noise conditional score network. $x_t = x_0 + \sigma_t \epsilon$, learn $s_\theta(x_t, \sigma_t)$ to estimate $\nabla_x \log p(x_t)$.

$x_t = \alpha_t x_0 + \sigma_t \epsilon$, score $s_\theta(x_t, t)$? MSE loss $L = \mathbb{E}_t \|\bar{s}_\theta(x_t, t) - \nabla_x \log p(x_t)\|^2$. $\mathbb{E}_t [\bar{s}_\theta(x_t, t)^\top \nabla_x \log p(x_t)] = \mathbb{E}_t [\nabla \cdot \bar{s}_\theta(x_t, t)]$, using $\nabla \cdot (sp) = (\nabla \cdot s)p + s \cdot \nabla p$

Denoisong score matching: $\nabla_x \log p(x_t) = \mathbb{E}_{x_0 \sim p(\cdot | x_t)} [\nabla_x \log p(x_t | x_0)]$

$$\mathbb{E}_{x_t} \|\bar{s}_\theta(x_t, t) - \nabla_x \log p(x_t | x_0)\|^2,$$

$L_{\text{DSM}} = \mathbb{E}_{x_0 \sim p_{\text{data}}, t \sim [0, 1], x_t \sim q(x_t | x_0)} \|\bar{s}_\theta(x_t, t) - \nabla_x \log q(x_t | x_0)\|^2$, $q(x_t | x_0)$ is tractable. If $x_t = \alpha_t x_0 + \sigma_t \epsilon$, $\nabla_x \log q(x_t | x_0) = \nabla_{x_t} (-\frac{(x_t - \alpha_t x_0)^2}{2\sigma_t^2}) = -\frac{x_t - \alpha_t x_0}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t}$. Let $S_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sigma_t}$, loss same as DDPM.

SDE: $dx = f(x, t)dt + g(t)dW_t$,
reverse: $dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)]dt + g(t)d\tilde{W}_t$.
A equivalent ODE (same distribution when same t): Probability-flow ODE:
 $dx = \left[f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x) \right] dt$. Likelihood along ODE: $\log p_0(x_0) = \log p_T(x_T) + \int_0^T \text{tr} \left(\frac{\partial f_t}{\partial x_t} \right) dt$.
Inference: use learned score to solve SDE/ODE using a solver. Hence
#sample-steps in training and inference can be different.
DDIM: sample according to schedule

5.3 DPM-Solver and Applications

DPM solver 1 order DPM = DDIM.
DPM-Solver integrates diffusion ODE with high-order exponential integrators, reducing sampling steps. **Latent diffusion**: diffuse in autoencoder latent space, cheaper than pixel space. **DiT/SORA**: transformer over spacetime latent patches. **DALL-E 2**: CLIP prior + diffusion decoder. **DALL-E 3**: synthetic captions improve prompt following. **SDEdit**: add noise to input image then denoise under prompt. **ControlNet**: extra condition branch injects edges/depth/pose. **Logit-normal sampling**: biases timestep sampling toward informative middle region.

5.4 Flow Matching

Continuous flow $dx_t/dt = v_\theta(x_t, t)$. Objective: $\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, x_t} \|v_\theta(x_t, t) - u_t(x_t)\|^2$. Conditional path from noise x_0 to data x_1 : $x_t = (1-t)x_0 + tx_1$, target $u_t = x_1 - x_0$. Conditional Flow Matching (CFM) samples pairs and regresses conditional velocity; marginal vector field is recovered in expectation. **Non-uniqueness**: many vector fields induce same marginals; choice affects training variance and sampling trajectory. **SD3**: flow target often parameterizes velocity between noise/data under continuous timestep.

6 L8–9 Sequence Modeling

6.1 Sequence Data and Tokenization

Sequence data includes language, audio, action; images/DNA/protein can be serialized. Autoregressive factorization: $p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{<t})$. **Tokenization**: text $\rightarrow$ basic units $\rightarrow$ token IDs. Byte-Pair Encoding (BPE) repeatedly merges frequent adjacent symbols; balances vocabulary size and sequence length. **Embedding**: token ID $i \rightarrow e_i = W_E[i]$; positional info is needed because token embeddings alone are order-free.

6.2 Autoregressive CNN and Pixel Models

WaveNet: causal/dilated temporal convolution expands receptive field without recurrence. **PixelCNN**: image likelihood $p(x) = \prod_i p(x_i | x_{<i})$. under raster order; masked convolution prevents access to future pixels. Gated PixelCNN uses gates to improve expressiveness.

6.3 RNN and LSTM

RNN: $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$, $y_t = W_{hy}h_t + b_y$. BPTT multiplies many Jacobians; singular values < 1 vanish, > 1 explode.
LSTM: $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$, $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$, $\tilde{c}_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$, $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$, $h_t = o_t \odot \tanh(c_t)$. Additive cell path gives controllable memory; forget gate decides retention.

6.4 Attention and Transformer

For token t : $q_t = W_Q x_t$, $k_j = W_K x_j$, $v_j = W_V x_j$, $\alpha_{tj} = \frac{\exp(q_t^\top k_j / \sqrt{d_k})}{\sum_i \exp(q_t^\top k_i / \sqrt{d_k})}$, $y_t = \sum_j \alpha_{tj} v_j$. Matrix form: $\text{Attn}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$.

Multi-head: parallel attention subspaces, concatenate and project. **Causal mask**: block future tokens. **Transformer block**: attention + MLP + residual + normalization. Complexity $O(T^2 d)$; FlashAttention reduces memory IO, Mamba/SSM explores linear-time sequence mixing.

6.5 Objectives and Case Studies

Objectives: autoregressive LM, masked/denoising, contrastive predictive. **Recipe**: pretraining, fine-tuning, low-rank tuning. **GPT**: decoder-only causal Transformer; next-token pretraining gives in-context learning with scale. **ViT**: image patches as tokens. **Swin**: shifted local windows for hierarchical vision.

CLIP: contrastive image-text alignment,

$$L = -\frac{1}{2} \left[\frac{1}{N} \sum_i \log \frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ij}/\tau}} + \frac{1}{N} \sum_i \log \frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ji}/\tau}} \right].$$

SigLIP: pairwise sigmoid loss avoids global softmax. **Whisper**: audio Transformer trained on large-scale speech. **MAR**: masked autoregressive modeling for images.

7 L10 Modern Large Models

7.1 Pretraining and Scaling

Autoregressive LM: maximize $\sum_t \log p_\theta(x_t | x_{<t})$. Base model learns continuation; chat model learns instruction-following behavior through post-training. **Scaling laws**: loss decreases smoothly with model size N , data D , compute C , often power-law: $L(N) \approx L_\infty + aN^{-\alpha}$, $L(D) \approx L_\infty + bD^{-\beta}$. Compute-optimal training balances parameters and tokens; not simply largest model on fixed data. Larger models reach the same loss with fewer samples / fewer optimization steps. **Prompting**: zero-shot/few-shot describes task in context; no parameter update. **Instruction tuning**: supervised fine-tuning on instruction-response mixtures.

7.2 Post Training

Using human feedback. **SFT**: next-token imitation on human demonstrations. It cannot express preference among multiple valid answers. **RLHF**: human preference. Can reward uncertainty awareness, e.g. "I don't know" when question is unanswerable. Negative samples provide learning signal unavailable in pure SFT. Helps eliminating hallucination. **Reward model**: train from pairwise rankings:

$$L_{\text{RM}} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

Policy optimization with KL:

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot | x)} [r_\phi(x, y)] - \beta \text{KL}(\pi(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)).$$

KL prevents policy from drifting too far from SFT/reference.

7.3 RAG and Reasoning

RAG: retrieve documents d then generate:

$$p(y|x) \approx \sum_{d \in \mathcal{R}(x)} p_\theta(y|x, d) p(d|x).$$

Retrieval supplies missing knowledge but does not automatically solve multi-step reasoning. **Inference-time scaling**: spend more test-time compute on search, verification, self-consistency, or long reasoning trajectories. For reasoning model:

$$\max_{\pi} \mathbb{E}_{\tau \sim \pi(\cdot | x)} [R(\text{answer}(\tau), y)] - \beta \text{KL}(\pi \| \pi_{\text{ref}}).$$

Answer verifier gives sparse reward; RL can optimize thinking trajectory even without unique chain-of-thought labels.

7.4 Multimodal Models

Vision-language few-shot: convert image/video tasks to text generation conditioned on visual tokens. **Flamingo**: frozen vision encoder + frozen LM with cross-attention layers and Perceiver resampler for interleaved image/video-text. **LLaVA**: CLIP vision encoder + projection to Vicuna/LLM; visual instruction tuning teaches chat-style responses. **Unified multimodal model**: one decoder handles understanding, generation, editing, reasoning; may use separate experts/tokenizers for text vs image. **BAGEL-style idea**: decoder-only unified model with modality-specific experts; understanding often emerges before reasoning/editing, which need larger scale.

8 Cross-Lecture Map

Density models: EBM defines $p \propto e^{-E}$ but sampling is hard; flow gives exact likelihood via invertible map; VAE gives ELBO; diffusion learns reverse denoising; flow matching learns vector field directly. **Implicit models**: GAN learns sampler without tractable likelihood. **Sequence models**: Transformer turns all modalities into token sequences. **Large models**: pretraining builds capability, post-training aligns behavior, inference-time compute and multimodal grounding extend usability.